Android 快速开机
启动优化指南

版本号: 1.0
发布日期: 2021.01.26

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2021.01.26	AWA0681	文档初始版本

广州星翼电子科技有限公司zdyzlinux

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1

2 Android P 开机启动分解 **2**
2.1 BootLoader 2
2.1.1 SBOOT 2
2.1.2 UBOOT 2
2.2 kernel 2
2.3 Android 3

3 开机启动优化调试工具 **4**
3.1 bootchart 4
3.2 systrace 6
3.3 aw bootevent 7
3.4 logcat 7
3.4.1 logcat 分析启动的关键字 7

4 开机启动优化常见措施 **10**
4.1 bootLoader 优化常见措施 10
4.1.1 优化 usb burn key 检测 10
4.1.2 减少 bootloader 打印 10
4.2 kernel 优化常见措施 11
4.2.1 内核模块的裁剪 11
4.2.2 优化多款 sensor 启动耗时 11
4.2.3 提高 emmc/flash 读写性能 11
4.2.4 文件系统的选择 12
4.2.5 initcall 并行化 13
4.3 Android 优化常见措施 14
4.3.1 调整 IO 参数 14
4.3.2 使用简单的动画 14
4.3.3 提前结束动画 14
4.3.4 异步加载不紧急的 server 14
4.3.5 延迟启动不紧急的 systemui 15
4.3.6 延迟 SystemBars 的 camerainit 时机 16
4.3.7 限制 phone 为非 persistent 应用 16
4.3.8 延迟场景管理 BOOTCOMPLETE 的发送时机 17
4.3.9 去除 statusbar 18
4.3.10 删除 gms 包 18
4.3.11 UDISK 分区的文件系统类型 18
4.3.12 使用简单的 launcher 19

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3.13 减少不必要的预装 apk 19

广州星翼电子科技有限公司zdyzlinux

插　图

3-1 bootchart 5
3-2 boot_trace 7
3-3 SystemUIBootTiming 8
4-1 sys_config.fex 10
4-2 debug_mode 10
4-3 systemui_start 16
4-4 systembars 16
4-5 statusbar 18

广州星翼电子科技有限公司zdyzlinux

1 概述

1.1 编写目的

- 本文档提供 Android 快速开机启动优化指南
- 本文档提供的快速排查指南工程思路如下：启动分解 + 调式工具 + 常见优化措施
- 本文档仅用于帮助开发人员快速排查和解决初级问题问题，如果按照文档中的指导仍无法解决问题，请联系全志原厂 FAE 进一步协助解决，并提供说明是否按照文档中的排查方案进行过必要的排查，提供排查过程的必要记录和数据

1.2 适用范围

基于 uboot2014，Android P，A50 安全方案平台编写，适用于 Android P 以后的方案。

1.3 相关人员

本文档主要适用于以下工程师：系统工程师，性能优化工程师，主要面对开机启动优化方面的需求

广州星翼电子科技有限公司zdyzlinux

2 Android P 开机启动分解

Android P 的启动大体可以分为 BootLoader、kernel、Android 三大阶段

2.1 BootLoader

BootLoader 阶段主要包含 2 个阶段：

2.1.1 SBOOT

1. rsb 通讯以及 pmu 初始化
2. rtc 初始化
3. dram 初始化
4. emmc 初始化
5. 加载并校验 toc1
6. scp 以及 optee 的初始化
   这部分比较耗时的是 toc1 加载校验以及 scp 和 optee 的初始化，大概 400ms。

2.1.2 UBOOT

1. PMU 初始化
2. 多核启动
3. disp 与 emmc 初始化
4. Secure storage 初始化
5. Usb burn key
6. Verified boot 的加载与校验
7. 启动 kernel

2.2 kernel

1. Initcall 类

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. Platform driver 类
3. Platform device 类
4. Probe 总线类

2.3 Android

1. Init
2. Native、Hal
3. Zygote
4. SystemServer
5. SystemUi
6. Launcher

广州星翼电子科技有限公司zdyzlinux

3 开机启动优化调试工具

3.1 bootchart

bootchart 是一个图形化的开机启动调式工具，它能在系统启动过程自动收集 CPU 占用率、IO 占用率、进占用等信息，最终以图形化的形式展现出来。

bootchart 需要内核支持 ftrace debug，在 Android P 已经支持的很好，很方便就能够使用，下面是方法步骤介绍：

1. touch /data/bootchart/enabled
2. reboot
3. cd /data/bootchart
4. tar czf bootchart.tgz *
5. adb pull /data/bootchart/bootchart.tgz
6. 把 bootchart.jar 和 bootchar.tgz 放在同一目录
7. 在 cmd 里执行 java -jar bootchart.jar bootchart.tgz
8. 按照以上步骤就会生成 bootchatr.png，如图 [3-1]

广州星翼电子科技有限公司zdyzlinux

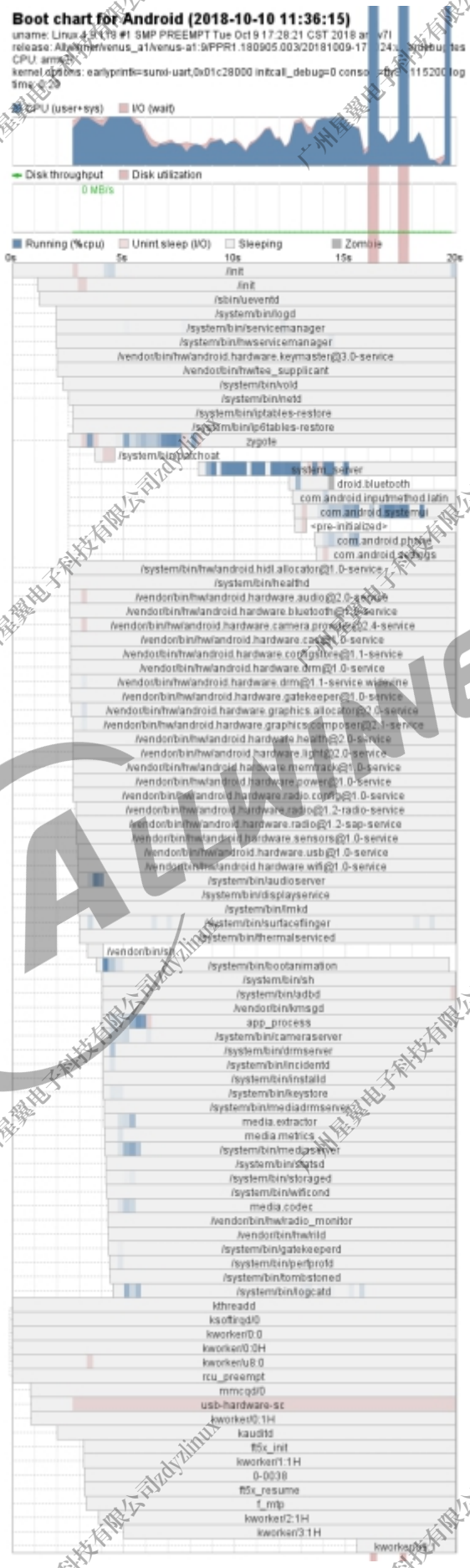

图 3-1：bootchart

3.2 systrace

systrace 是 Google 支持的一款工具，用于系统启动过程中收集 kernel 以及 Android 的 trace 信息，用于分析开机启动。

systrace 同样也需要内核支持 ftrace debug，在 Android P 已经支持的很好，很方便就能够使用，下面是方法步骤介绍：

- 进入 frameworks/native/cmds/atrace/atrace.rc 注释下面这句：

```
write /sys/kernel/debug/tracing/tracing_on 0
```

- 进入方案目录，修改 BoardConfig.mk 文件

```
BOARD_KERNEL_CMDLINE := ... trace_buf_size=64M
trace_event=sched_wakeup,sched_switch,sched_blocked_reason,sched_cpu_hotplug,block,squashfs
```

如果是 ext4 文件系统，则最后改为 ext4

- 在方案目录，修改 init.device.rc

```
on property:sys.boot_completed=1
    write /d/tracing/tracing_on 0
    write /d/tracing/events/ext4/enable 0
    write /d/tracing/events/block/enable 0
```

- 在方案目录，修改 venus_a1.mk（方案的 mk 文件）

```
PRODUCT_PROPERTY_OVERRIDES += debug.atrace.tags.enableflags=802922
```

以上修改完成后，重新编译固件，烧录后，采取第二次以后的 systrace 用于分析，步骤：

1. cat /d/tracing/trace > /data/boot_trace
2. 将生成的 boot_trace adb pull 出来，放到 Android N 的 SDK 的/external/chromium-trace/catapult/tracing/bin/路径下，原因是要使用 trace2html 工具，这个工具在 Android O 后并没有支持。
3. ./trace2html boot_trace –output boot_trace.html 生成 boot_trace.html
   将生成的 boot_trace.html 拿出来，使用 chrome 浏览器打开，打开后如图 [3-2]：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

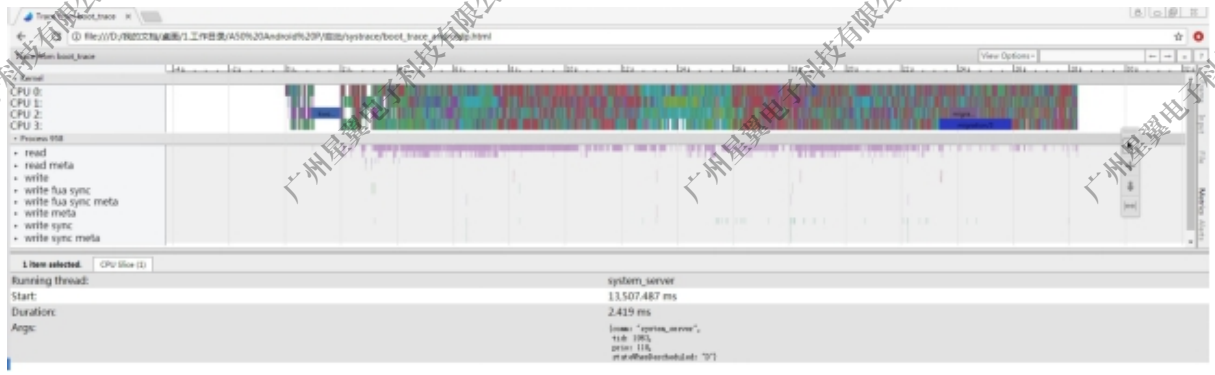

图 3-2: boot_trace

可通过键盘 awsd 来放大缩小查看整个启动过程的信息。

3.3 aw bootevent

aw bootevent 是一个开机启动调式工具，通过它能够初步判断开机启动有哪些事件，哪些事件耗时多长时间，它是一个开机启动的量化工具，能够帮助工程师分析开机启动过程哪些模块需要优化，但它不能直接实现优化的效果，只能起到排查的作用，具体的优化工作还需要具体模块的工程师花时间阅读代码去优化。使用方法：cat /proc/bootevent

3.4 logcat

logcat 是最原始的分析调式工具，一般而言，通过 logcat > /data/logcat.txt，将整个启动 logcat 抓出来。还可以通过 logcat -b events | grep boot* 来抓取重要的 events

3.4.1 logcat 分析启动的关键字

- 搜索"phase"，来判断启动是否完成，如果到了 1000 表示开机已经结束，如下：

```
console:/ # logcat | grep phase
11-07 10:41:22.751  1900  1900 I SystemServiceManager: Starting phase 100
11-07 10:41:26.843  1900  1900 I SystemServiceManager: Starting phase 480
11-07 10:41:26.893  1900  1900 I SystemServiceManager: Starting phase 500
11-07 10:41:27.122  1900  1900 I SystemServiceManager: Starting phase 520
11-07 10:41:27.163  1900  1900 I SystemServiceManager: Starting phase 550
11-07 10:41:28.023  1900  1900 I SystemServiceManager: Starting phase 600
11-07 10:41:33.564  1900  1900 I SystemServiceManager: Starting phase 1000
```

- 搜索"SystemServerTiming"，分析 system server 耗时，如下：

广州星翼电子科技有限公司zdyzlinux

```
console:/ # logcat | grep SystemServerTiming
11-07 10:41:22.133  1900  1900 D SystemServerTiming: InitBeforeStartServices took to
    complete: 606ms
11-07 10:41:22.142  1900  1900 D SystemServerTiming: ReadingSystemConfig took to complete:
    8ms
11-07 10:41:22.156  1900  1900 D SystemServerTiming: StartInstaller took to complete: 14ms
11-07 10:41:22.159  1900  1900 D SystemServerTiming: DeviceIdentifiersPolicyService took to
     complete: 2ms
11-07 10:41:22.699  1900  1900 D SystemServerTiming: StartActivityManager took to complete:
     539ms
11-07 10:41:22.718  1900  1900 D SystemServerTiming: StartPowerManager took to complete: 19
    ms
11-07 10:41:22.722  1900  1900 D SystemServerTiming: InitPowerManagement took to complete:
    4ms
11-07 10:41:22.724  1900  1900 D SystemServerTiming: StartRecoverySystemService took to
    complete: 2ms
11-07 10:41:22.730  1900  1900 D SystemServerTiming: StartLightsService took to complete: 2
    ms
11-07 10:41:22.738  1900  1900 D SystemServerTiming: StartSidekickService took to complete:
     0ms
11-07 10:41:22.751  1900  1900 D SystemServerTiming: StartDisplayManager took to complete:
    13ms
11-07 10:41:22.784  1900  1900 D SystemServerTiming: WaitForDisplay took to complete: 33ms
11-07 10:41:25.000  1900  1993 D SystemServerTimingAsync: AppDataFixup took to complete:
    155ms
11-07 10:41:25.433  1900  1900 D SystemServerTiming: StartPackageManagerService took to
    complete: 2648ms
11-07 10:41:25.437  1900  1900 D SystemServerTiming: StartOtaDexOptService took to complete
    : 2ms
11-07 10:41:25.441  1900  1900 D SystemServerTiming: StartUserManagerService took to
    complete: 3ms
11-07 10:41:25.442  1900  1900 D SystemServerTiming: InitAttributerCache took to complete:
    1ms
11-07 10:41:25.460  1900  1900 D SystemServerTiming: SetSystemProcess took to complete: 18
    ms
......
```

- 搜索"SystemUIBootTiming"，分析 system ui 耗时，如下：

```
console:/ # logcat | grep SystemUIBootTiming
11-07 10:41:28.055  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.Dependency took to complete: 26ms
11-07 10:41:28.095  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.util.NotificationChannels took to complete: 40ms
11-07 10:41:28.105  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.statusbar.CommandQueue$CommandQueueStart took to complete: 9ms
11-07 10:41:28.485  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.keyguard.KeyguardViewMediator took to complete: 380ms
11-07 10:41:29.708  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.recents.Recents took to complete: 1222ms
11-07 10:41:29.774  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.volume.VolumeUI took to complete: 32ms
11-07 10:41:30.008  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.stackdivider.Divider took to complete: 217ms
11-07 10:41:31.864  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.SystemBars took to complete: 1853ms
11-07 10:41:31.876  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.usb.StorageNotification took to complete: 11ms
11-07 10:41:31.888  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.power.PowerUI took to complete: 12ms
11-07 10:41:31.891  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.media.RingtonePlayer took to complete: 3ms
11-07 10:41:31.893  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.keyboard.KeyboardUI took to complete: 2ms
11-07 10:41:31.895  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.pip.PipUI took to complete: 2ms
11-07 10:41:31.898  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.shortcut.ShortcutKeyDispatcher took to complete: 3ms
11-07 10:41:31.899  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.VendorServices took to complete: 1ms
11-07 10:41:31.902  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.util.leak.GarbageMonitor$Service took to complete: 3ms
11-07 10:41:31.905  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.LatencyTester took to complete: 3ms
11-07 10:41:31.910  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.globalactions.GlobalActionsComponent took to complete: 4ms
11-07 10:41:31.917  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.ScreenDecorations took to complete: 6ms
11-07 10:41:31.919  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.fingerprint.FingerprintDialogImpl took to complete: 1ms
11-07 10:41:31.921  2104  2104 D SystemUIBootTiming: StartServicescom.android.systemui.SliceBroadcastRelayHandler took to complete: 2ms
11-07 10:41:31.921  2104  2104 D SystemUIBootTiming: StartServices took to complete: 3893ms
```

图 3-3: SystemUIBootTiming

- 搜索"StartServices took to complete"，来判断 system server 以及 systemui 整个启动加载总耗时，如下：

```
console:/ # logcat | grep "StartServices took to complete"
11-07 10:41:22.133  1900  1900 D SystemServerTiming: InitBeforeStartServices took to
    complete: 606ms
11-07 10:41:28.950  1900  1900 D SystemServerTiming: StartServices took to complete: 6816ms
11-07 10:41:31.921  2104  2104 D SystemUIBootTiming: StartServices took to complete: 3893ms
```

广州星翼电子科技有限公司zdyzlinux

4 开机启动优化常见措施

4.1 bootLoader 优化常见措施

4.1.1 优化 usb burn key 检测

默认 SDK 需要打开 burn_key，导致每次开机都需要检查 usb 是否存在，假设 usb 不存在，当前整个 usb 检查过程耗时 800ms，原因是检测 usb 的方法实现有问题，不能真正判断 usb 是否存在，而导致了延时的存在。修复检测 usb 的方法后，可以优化开机启动时间 800ms。

也可以将 sys_config.fex 的 burn_key 设置为 0。

```
;-----------------------------
[target]
boot_clock        = 1008
storage_type      = -1
burn_key          = 1
dragonboard_test  = 0
```

图 4-1: sys_config.fex

4.1.2 减少 bootloader 打印

将 sys_config.fex 的 debug_mode 改为 0 后，整个 uboot 的打印基本就没有了，主要是减少了 boot0 的打印，因为那会频率低，串口输出慢，开机时间节省大概 130ms。Debug_mode=1 是正常的发布打印等级，如果要看到更多的 uboot log，需要设置为 4，如果设置为 0，则开机启动时间能减少 130ms。

```
;--------------------
[platform]
eraseflag    = 1
next_work    = 3
debug_mode   = 1
```

图 4-2: debug_mode

目前这个修改没有提交，留给客户自己选择。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 kernel 优化常见措施

4.2.1 内核模块的裁剪

内核模块的裁剪分两部分，第一部分是 debug 模块，debug 主要是针对一些开机启动无关的 debug，例如 slub_debug，这里要说的是 tracers 的 debug，对于一些不需要过 GMS 的平台，针对发布的 SDK 把 tracers 关了能节省大概 500ms 的时间。由于 A50 的 CTS 测试要求要打开 tracers，这里保持不修改，但是对于车载产品是可以删除的。

```
Kernel hacking  --->
    [*] Tracers  --->  //把*去掉表示关掉tracers
```

第二部分指的是一些不必要的驱动模块，尤其是耗时的模块，可以通过 make menuconfig 的方法把不必要的硬件驱动去除，例如 A50 a3 有部分 sensor 是没有使用的，那么可以将未使用的模块不要编译进内核。

4.2.2 优化多款 sensor 启动耗时

通过 cat /proc/bootevent 发现内核阶段 sensor 耗时 1.3s，原因是有多个 sensor 同时启动，而且单个 sensor 超过 500ms，仔细阅读代码发现单个 sensor 启动耗时长的原因是因为代码里面有 msleep(500)，多个 sensor 一并启动的原因是因为相互之前没有关联，通过添加标志位来避免多个 sensor 同时启动的问题。最终优化 1.2s 的启动时间。

4.2.3 提高 emmc/flash 读写性能

A50 有 nand 和 emmc 两种 flash 介质，其中 emmc 还有高速模式（HS400-100MHz），通过实验发现，nand flash 介质对启动影响特别大，由于 nand 的连续读速度慢（ext4:20MB/s，squashfs：16MB/s），开机启动时间长达 60s 左右。而 emmc 即使在 SDR50 的模式下连续读（65MB/s），开机启动时间 30s 左右。当把 emmc 改为高速模式后，开机启动时间有大概 2.2s 的优化，主要原因是整个启动过程读 system vendor 分区的速度快了，例如 zygote 加载 classes，pms scan 速度加快等。

修改 emmc 为高速的模式的补丁为：

```
--- a/pack/chips/sun8iw15p1/configs/a1/sys_config.fex
+++ b/pack/chips/sun8iw15p1/configs/a1/sys_config.fex
@@ -58,8 +58,8 @@ aldo5_vol         = 2500
 ;-----------------------------------------------------------------------------
 ;gpio_bias pc_bias for emmc high speed mode use
 ;-----------------------------------------------------------------------------
-;[gpio_bias]
-;pc_bias         = "axp858:bldo5:1800"
+[gpio_bias]
+pc_bias         = "axp858:bldo5:1800"
```

```
 [card_boot]
  logical_start    = 40960
@@ -100,7 +100,7 @@ sdc_d7          = port:PC16<3><1><3><default>
  sdc_emmc_rst     = port:PC01<3><1><3><default>
  sdc_ds           = port:PC00<3><1><3><default>
  sdc_ex_dly_used = 2
-sdc_io_1v8 = 0
+sdc_io_1v8 = 1
```

4.2.4 文件系统的选择

Android P 开始，Google 推荐 system 和 vendor 采用 squashfs 文件系统，它是一个压缩的只读文件系统，Google 从两个角度选择了它。一是可以减少 system 和 vendor 分区大小，对于 16GB flash 的设备可以预留多一些空间给用户定制；二是 Google 想推只读的 system/vendor 分区，这是从安全的机制考虑。

但是 squashfs 文件系统的缺点是压缩的，每次读内容都需要解压，当然就会有所损耗，Google 官网说明对性能影响是 2%，但是实测开机启动速度，对于 emmc 设备，有 3.5s 的影响，影响超过 10%。对于存储设备大或者没有文件系统要求的产品，可以选择 ext4 文件系统，例如车载产品。这样可以节省 3.5s 的开机启动时间。

A50 Android P 默认的 SDK 是采用 squashfs 文件系统，如需改为 ext4，需修改：

内核目录：

```
--- a/arch/arm/boot/dts/sun8iw15p1.dtsi
+++ b/arch/arm/boot/dts/sun8iw15p1.dtsi
@@ -76,7 +76,7 @@
                                        mnt_flags = "ro,barrier=1";
                                        name = "vendor";
                                        status = "ok";
-                                       type = "squashfs";
+                                       type = "ext4";
                                };
                        };
                };
```

方案目录

```
--- a/BoardConfig.mk
+++ b/BoardConfig.mk
@@ -20,13 +20,13 @@ BOARD_BUILD_SYSTEM_ROOT_IMAGE := true
 TARGET_RECOVERY_FSTAB := device/softwinner/venus-a1/recovery.fstab

 # Enable SquashFS for /system
-BOARD_SYSTEMIMAGE_FILE_SYSTEM_TYPE := squashfs
+BOARD_SYSTEMIMAGE_FILE_SYSTEM_TYPE := ext4
 BOARD_SYSTEMIMAGE_SQUASHFS_COMPRESSOR := lz4
 BOARD_SYSTEMIMAGE_SQUASHFS_BLOCK_SIZE := 65536
 BOARD_SYSTEMIMAGE_SQUASHFS_COMPRESSOR_OPT := -Xhc

 # # Enable SquashFS for /vendor
-BOARD_VENDORIMAGE_FILE_SYSTEM_TYPE := squashfs
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
+BOARD_VENDORIMAGE_FILE_SYSTEM_TYPE := ext4
 BOARD_VENDORIMAGE_SQUASHFS_COMPRESSOR :=  lz4
```

4.2.5 initcall 并行化

目的是将 module_init 的模块并行化处理，这个方法在 v 系列的产品中有起到一定的作用，但是经过大量实验，在 A50 上并没有得到明显的改善。有了多线程处理 module_init，还需要观察自己平台哪些模块可以并行处理，例如 A50 平台，通过 cat /proc/bootevent，或者通过 initcall_debug 开关查看哪些模块可以并行处理。举个例子：

```
--- a/drivers/usb/host/ehci-hcd.c
+++ b/drivers/usb/host/ehci-hcd.c
@@ -1405,7 +1405,7 @@ static int __init ehci_hcd_init(void)
        clear_bit(USB_EHCI_LOADED, &usb_hcds_loaded);
        return retval;
 }
-module_init(ehci_hcd_init);
+device_paralell_initcall(ehci_hcd_init);

 static void __exit ehci_hcd_cleanup(void)
 {
diff --git a/drivers/usb/host/ohci-hcd.c b/drivers/usb/host/ohci-hcd.c
index 3711230..c3d38f3 100644
--- a/drivers/usb/host/ohci-hcd.c
+++ b/drivers/usb/host/ohci-hcd.c
@@ -1358,7 +1358,7 @@ static int __init ohci_hcd_mod_init(void)
        clear_bit(USB_OHCI_LOADED, &usb_hcds_loaded);
        return retval;
 }
-module_init(ohci_hcd_mod_init);
+device_paralell_initcall(ohci_hcd_mod_init);
 static void __exit ohci_hcd_mod_exit(void)
 {
diff --git a/drivers/video/fbdev/sunxi/disp2/disp/dev_disp.c b/drivers/video/fbdev/sunxi/
    disp2/disp/dev_disp.c
index 9be2a1e..ceda78a 100644
--- a/drivers/video/fbdev/sunxi/disp2/disp/dev_disp.c
+++ b/drivers/video/fbdev/sunxi/disp2/disp/dev_disp.c
@@ -3696,7 +3696,7 @@ static void __exit disp_module_exit(void)
        cdev_del(my_cdev);
 }

-module_init(disp_module_init);
+device_paralell_initcall(disp_module_init);
```

通过实验发现，修改后有大概几十 ms 的优化，优化效果不明显，所以没有合并到 A50 Android P 的 SDK。

但是如果在 nand 的设备，通过 cat /proc/bootevent 发现有大量 kernel 阶段的模块启动时间很长，且关联性不大的时候，可以考虑使用 initcall 并行化，这样能起到一定的优化效果。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3 Android 优化常见措施

4.3.1 调整 IO 参数

通过开机启动的过程中适当调整 io 参数，有助于 IO 速度的提升，最终帮助开机启动速度。
在方案的 rc 文件添加下面修改：

```
on late-fs
    write /sys/devices/platform/soc/sdc2/mmc_host/mmc0/mmc0:0001/block/mmcblk0/queue/
    iostats 0
    write /sys/devices/platform/soc/sdc2/mmc_host/mmc0/mmc0:0001/block/mmcblk0/queue/
    nr_requests 256
    write /sys/devices/platform/soc/sdc2/mmc_host/mmc0/mmc0:0001/block/mmcblk0/queue/
    read_ahead_kb 1024
on property:sys.boot_completed=1
    write /sys/devices/platform/soc/sdc2/mmc_host/mmc0/mmc0:0001/block/mmcblk0/queue/
    iostats 1
    write /sys/devices/platform/soc/sdc2/mmc_host/mmc0/mmc0:0001/block/mmcblk0/queue/
    nr_requests 128
    write /sys/devices/platform/soc/sdc2/mmc_host/mmc0/mmc0:0001/block/mmcblk0/queue/
    read_ahead_kb 128
```

4.3.2 使用简单的动画

A50 Android P 默认使用的动画是比较复杂的，里面含有大量的图片，从 bootchart 中可看到 bootanimation 会占用一些启动时间，如果某些平台对开机动画没有那么强烈的要求，可以裁剪一些动画图片，甚至不启动动画服务，可以节省 1.8s 左右。

修改：可将/system/media/bootanimation.zip 推出来修改，将 part1 和 part2 的图片删减大部分，然后重新推到上述路径。（前提是 ext4 文件系统，因为 squashfs 是只读的，不允许修改）

4.3.3 提前结束动画

开机动画退出信号意味着系统开机启动的完成，所以早点结束动画能够优化启动时间，Android P 上，我们在 LaunchTimeLocked 阶段提前设置动画退出的属性，有利于加速 launcher 的启动。优化效果 1.1s，已经集成补丁。

4.3.4 异步加载不紧急的 server

通过下面命令，可以查看开机过程中，system server 启动的详细过程，可以知道哪些服务比较耗时。

```
logcat | grep SystemServerTiming
```

其实，Google 已经对 system server 进行优化过了，采用的是一个 system 线程池的方法，将不紧急的任务进行异步处理。原生补丁如下：

```
路径: frameworks/base/
commit e29a5a11529dc7df82911b48b9f95461383cbcc2
Author: Fyodor Kupolov <fkupolov@google.com>
Date:   Fri Dec 16 16:14:17 2016 -0800

    Added SystemServerInitThreadPool

    System services can use it during the boot to submit tasks that can be run in
    parallel with the main thread.

    Switched PersistentDataBlockService and FingerprintService from FgThread to
    the new thread pool.

    UiModeManagerService: update initial configurations on init thread. They run
    while holding the mLock so no extra synchronization barriers are needed at a
    later stage.

    Test: manual - device boots without errors
    Test: ParallelPackageParserTest passes
    Change-Id: I548f34b0a18f61924e09a39afb12e085cde35442
```

通过线程池的方法，能够将某些服务进行异步处理，如：

```
1-02 14:09:44.039  1890  1993 D SystemServerTimingAsync: AppDataFixup took to complete: 146
    ms
11-02 14:09:44.550  1890  2003 D SystemServerTimingAsync: StartSensorService took to
    complete: 35ms
11-02 14:09:44.629  1890  1952 D SystemServerTimingAsync: SecondaryZygotePreload took to
    complete: 34ms
11-02 14:09:44.925  1890  2003 D SystemServerTimingAsync: StartHidlServices took to
    complete: 5ms
11-02 14:09:45.943  1890  1993 D SystemServerTimingAsync: AppDataPrepare took to complete:
    1903ms
11-02 14:09:46.405  1890  2003 D SystemServerTimingAsync: WebViewFactoryPreparation took to
     complete: 51ms
11-02 14:09:46.989  1890  1952 D SystemServerTimingAsync: StartWatchdog took to complete: 3
    ms
```

启动，StartWatchdog 是经过修改的，原本大概需要 100ms

```
10-19 11:21:59.064  1971  1971 I SystemServer: StartWatchdog
10-19 11:21:59.172  1971  1971 D SystemServerTiming: StartWatchdog took to complete: 108ms
```

4.3.5 延迟启动不紧急的 systemui

开机启动过程中，最后环节比较耗时的就是 systemui 的启动了，整个过程长达 4s 左右，其中 SystemBars 耗时最长，接近 2s。

通过 logcat | grep SystemUIBootTiming 可查看过程：

```
console:/ # logcat | grep SystemUIBootTiming
11-06 14:38:55.176  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.Dependency took to complete: 57ms
11-06 14:38:55.206  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.util.NotificationChannels took to complete: 26ms
11-06 14:38:55.213  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.statusbar.CommandQueue$CommandQueueStart took to complete: 6ms
11-06 14:38:56.013  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.keyguard.KeyguardViewMediator took to complete: 799ms
11-06 14:38:56.695  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.recents.Recents took to complete: 681ms
11-06 14:38:56.714  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.volume.VolumeUI took to complete: 4ms
11-06 14:38:56.902  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.stackdivider.Divider took to complete: 188ms
11-06 14:38:59.137  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.SystemBars took to complete: 2232ms
11-06 14:38:59.149  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.usb.StorageNotification took to complete: 11ms
11-06 14:38:59.161  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.power.PowerUI took to complete: 11ms
11-06 14:38:59.164  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.media.RingtonePlayer took to complete: 3ms
11-06 14:38:59.166  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.keyboard.KeyboardUI took to complete: 1ms
11-06 14:38:59.168  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.pip.PipUI took to complete: 1ms
11-06 14:38:59.171  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.shortcut.ShortcutKeyDispatcher took to complete: 4ms
11-06 14:38:59.172  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.VendorServices took to complete: 0ms
11-06 14:38:59.175  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.util.leak.GarbageMonitor$Service took to complete: 4ms
11-06 14:38:59.178  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.LatencyTester took to complete: 3ms
11-06 14:38:59.183  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.globalactions.GlobalActionsComponent took to complete: 4ms
11-06 14:38:59.191  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.ScreenDecorations took to complete: 8ms
11-06 14:38:59.193  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.fingerprint.FingerprintDialogImpl took to complete: 1ms
11-06 14:38:59.195  2095  2095 D SystemUIBootTiming: StartServicescom.android.systemui.SliceBroadcastRelayHandler took to complete: 2ms
11-06 14:38:59.196  2095  2095 D SystemUIBootTiming: StartServices took to complete: 4077ms
```

图 4-3: systemui_start

经过大量的实验发现，那么多 systemui services，能调整的只有 volume，其他都有依赖，一旦调整，系统就会出现开机异常，延迟 volume 后能够优化 200ms。

4.3.6 延迟 SystemBars 的 camerainit 时机

通过 logcat 可以发现 systemui 中的 SystemBars 特别耗时，长达 2s 的时间，通过分析源码结构发现 FlashlightController（闪光灯）也是在这里初始化，而部分平台甚至都没有闪光灯，所以考虑将这部分代码做一个延迟启动，这样可以加快 SystemBars 的启动时间，经优化能够节省大概 400ms 的时间。优化后的打印如下：

```
console:/ # logcat | grep SystemUIBootTiming
11-08 17:18:38.260  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.Dependency took to complete: 71ms
11-08 17:18:38.284  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.util.NotificationChannels took to complete: 23ms
11-08 17:18:38.289  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.statusbar.CommandQueue$CommandQueueStart took to complete: 4ms
11-08 17:18:39.095  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.keyguard.KeyguardViewMediator took to complete: 806ms
11-08 17:18:39.789  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.recents.Recents took to complete: 692ms
11-08 17:18:39.792  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.volume.VolumeUI took to complete: 1ms
11-08 17:18:40.120  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.stackdivider.Divider took to complete: 271ms
11-08 17:18:41.600  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.SystemBars took to complete: 1478ms
11-08 17:18:41.615  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.usb.StorageNotification took to complete: 14ms
11-08 17:18:41.628  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.power.PowerUI took to complete: 12ms
11-08 17:18:41.632  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.media.RingtonePlayer took to complete: 4ms
11-08 17:18:41.634  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.keyboard.KeyboardUI took to complete: 2ms
11-08 17:18:41.636  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.pip.PipUI took to complete: 2ms
11-08 17:18:41.640  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.shortcut.ShortcutKeyDispatcher took to complete: 3ms
11-08 17:18:41.641  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.VendorServices took to complete: 0ms
11-08 17:18:41.645  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.util.leak.GarbageMonitor$Service took to complete: 4ms
11-08 17:18:41.648  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.LatencyTester took to complete: 3ms
11-08 17:18:41.653  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.globalactions.GlobalActionsComponent took to complete: 5ms
11-08 17:18:41.663  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.ScreenDecorations took to complete: 9ms
11-08 17:18:41.665  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.fingerprint.FingerprintDialogImpl took to complete: 2ms
11-08 17:18:41.668  2116  2116 D SystemUIBootTiming: StartServicescom.android.systemui.SliceBroadcastRelayHandler took to complete: 3ms
11-08 17:18:41.668  2116  2116 D SystemUIBootTiming: StartServices took to complete: 3478ms
```

图 4-4: systembars

如果相应的产品没有 camera 设备，应该将删除 camera hal、rc service、包扫描不扫描 camera apk、StatusBars 不执行 cameraInit。

4.3.7 限制 phone 为非 persistent 应用

通过 logcat 和 bootchart 可以看到 phone 在开机启动完成之前就起来了，原因是 phone 是一个 persistent 应用，如果将其设置为非 persistent 就可以避免开机启动完成之前就起来。对于

一些没有 phone 要求的产品可以考虑将其优化一下，或者延迟启动 phone，由于在 A50 上修改后发现 phone 会 crash，所以没有集成，但是修改后通过 bootchart 看到 phone 确实不存在了，且不影响开机启动，但是由于没有做过一些场景的测试（例如视频通话），没有提交代码。作为一个启动优化的参考而提出：

修改：

```
路径: packages/services/Telephony/
--- a/AndroidManifest.xml
+++ b/AndroidManifest.xml
@@ -194,7 +194,6 @@
     <uses-permission android:name="android.permission.PACKAGE_USAGE_STATS" />

     <application android:name="PhoneApp"
-            android:persistent="true"
             android:label="@string/phoneAppLabel"
             android:icon="@mipmap/ic_launcher_phone"
             android:allowBackup="false"
```

4.3.8 延迟场景管理 BOOTCOMPLETE 的发送时机

有些平台场景管理在收到 BOOTCOMOLETE 广播后会设置为 normal，相应的 cpu 频率会降低，会影响 launcher 的启动速度，为了保证最高频率加载 launcher，我们延迟场景管理 BOOTCOMOLETE 的发送时机。

```
--- a/services/core/java/com/android/server/am/ActivityManagerService.java
+++ b/services/core/java/com/android/server/am/ActivityManagerService.java
@@ -791,6 +791,8 @@ public class ActivityManagerService extends IActivityManager.Stub

     final AppWarnings mAppWarnings;

+    private boolean bFinishFlag=false;
+
     /**
      * Dump of the activity state at the time of the last ANR. Cleared after
      * {@link WindowManagerService#LAST_ANR_LIFETIME_DURATION_MSECS}
@@ -8083,14 +8085,24 @@ public class ActivityManagerService extends IActivityManager.Stub
         return mKeyguardController.isKeyguardLocked();
     }

+    private Handler delayHandler = new Handler();
+    private Runnable delayrunnable = new Runnable() {
+       @Override
+       public void run() {
+           Slog.d(TAG, "delay adjust cpufreq ");
+           PowerManager pm = (PowerManager) mContext.getSystemService(Context.
    POWER_SERVICE);
+           if(mLocalPowerManager != null) {
+               mLocalPowerManager.powerHint(PowerManager.BOOTCOMPLETE , 0);
+           }
+       }
+    };
+
     final void finishBooting() {
```

```
-        /*aw_code: add bootcomplete operation with power_hal
-                 *--author:jiangbin 0918
-                 */
-                 if(mLocalPowerManager != null) {
-                        mLocalPowerManager.powerHint(PowerManager.BOOTCOMPLETE,0);
-                   }
-                /*end*/
+         if(bFinishFlag == false){
+             bFinishFlag = true;
+             delayHandler.postDelayed(delayrunnable, 15000);
+         }
+
```

4.3.9 去除 statusbar

隐藏 StatusBar 可以节省 2s 的启动时间，具体修改如下：

```
--- a/overlay/frameworks/base/packages/SystemUI/res/values/config.xml
+++ b/overlay/frameworks/base/packages/SystemUI/res/values/config.xml
@@ -22,4 +22,11 @@
     <!-- disable the automatic dimming animation to save cpu cycles-->
     <bool name="config_navigation_bar_enable_auto_dim_no_visible_wallpaper">false</bool>

+    <!-- Empty stub status/navigation bar -->
+    <!-- com.android.systemui.statusbar.tv.TvStatusBar or com.android.systemui.statusbar.phone.StatusBar -->
+    <string name="config_statusBarComponent" translatable="false">com.android.systemui.statusbar.tv.TvStatusBar</string>
```

图 4-5: statusbar

如果产品没有必要有 StatusBars，可以去掉 StatusBar，但对于平板类是必须的。

4.3.10 删除 gms 包

有些平台不需要过 cts 测试，应当把 gms 包删除，带有 gms 包的系统将会在包扫描以及应用启动方面增加启动时间，在 A50 的 emmc 设备上测试，如果删除 gms 包，可以节省 1.9s 的开机时间。对于盒子、车载等类似产品，如果不需要过 CTS 测试的话，不应该带有 gms 包。

应当在方案目录的方案名字的 mk 文件下删除，如 venus_a1.mk 删除下面这句。

```
$(call inherit-product-if-exists, vendor/google/products/gms_go-mandatory.mk)
```

4.3.11 UDISK 分区的文件系统类型

T 系列因为没有电池，关机常常不是按 power 键关机，换句话说不是走正常的 reboot/poweroff 流程，而是直接掉电关机，这样会导致系统再次起来后，会在 mount 阶段多一个 check 流程，相当于修复文件系统，通过实验发现 Android P 引进的 f2fs 文件系统在 check 阶段耗时 2s，相比之下 ext4 在这方面比较好，只需要 0.7s。所以，针对 T 系列的产品，建议 UDISK 使用 ext4 文件系统。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3.12 使用简单的 launcher

原生 launcher3 比较复杂，如果针对产品不需要原生 launcher3，可以使用自己的 launcher，且可以通过设置 android:directBootAware="true" 属性来提前启动 launcher，从而绕过 directboot 机制导致的 launcher 启动时间晚的问题。

4.3.13 减少不必要的预装 apk

针对产品的需求来裁剪不必要的 apk，如果能裁剪掉原生大部分的 apk，能够节省 500ms。

广州星翼电子科技有限公司zdyzlinux

著作权声明

版权所有 ©2021 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。